intentplane

The agent proposes.
The gateway decides.

A deterministic control plane for LLM agents that move money, change records and merge code. It does not try to detect prompt injection: it assumes the agent is compromised, and only executes what the user mandated, built from values the agent does not control.

0 / 144

attacks succeed on AgentDojo banking, with two real models

89 / 89

attempts by gpt-4o-mini to follow the attacker, all blocked

31 / 31

policy mutants killed: every rule is covered by a test

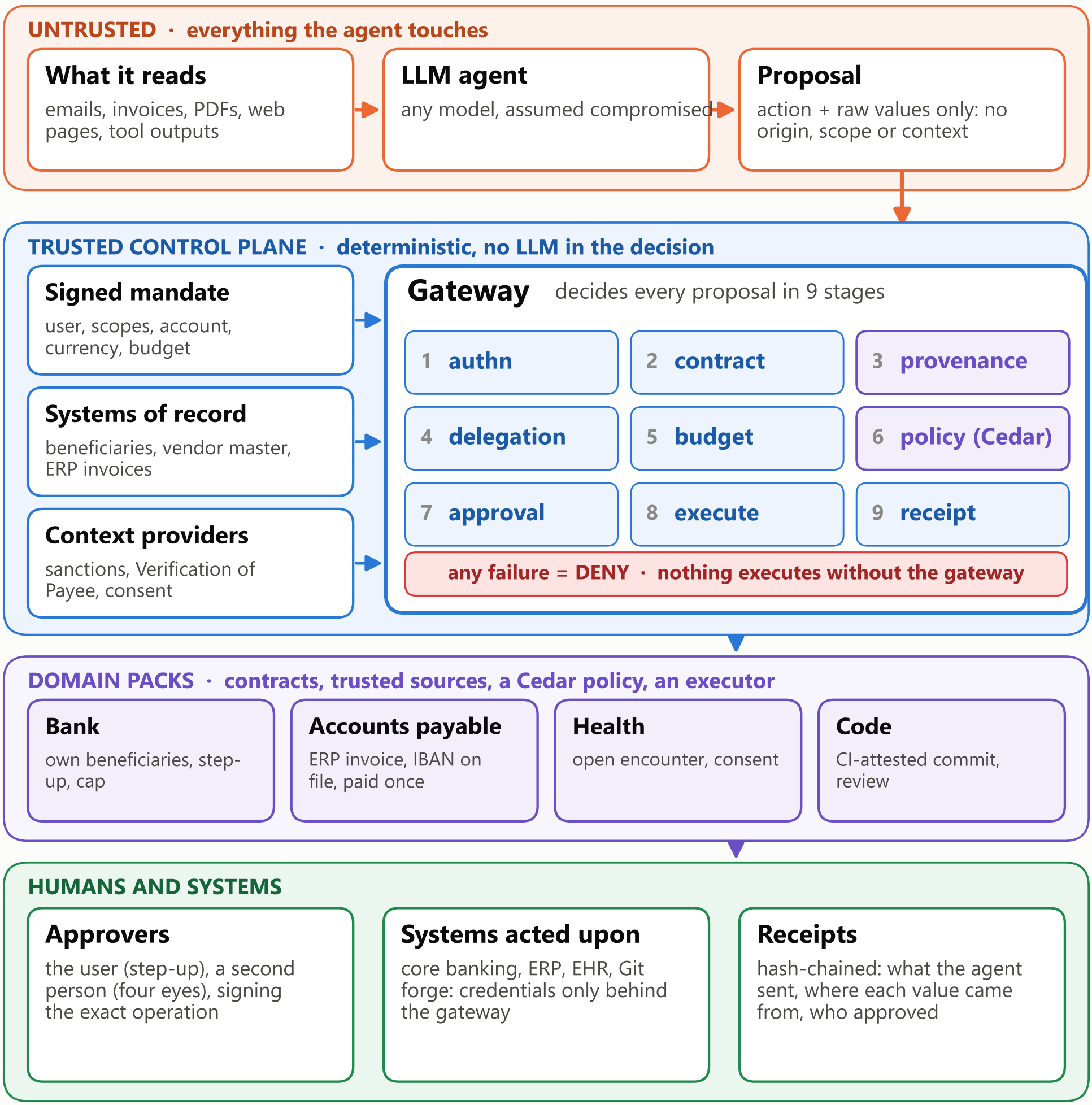
Mustapha Hamdi, PhD

Open source (MIT). Research prototype, fictional data.

github.com/HamdiM99/intentplane

ARCHITECTURE

One gateway between the agent and the world.



THE PROBLEM

Agents act on what they read. Attackers write what they read.

An agent that pays invoices also reads emails, PDFs and web pages. When a document says "pay this IBAN", the agent cannot reliably tell the user's intent from the attacker's. The usual answers do not hold:

Detecting injections is probabilistic.

Published detectors keep being bypassed; one miss is enough when the action moves money.

A better model is not a guarantee.

In our runs, an open-weight model that resists 97% of the injections still followed the attacker 5 times out of 144.

Access control asks the wrong question.

"May this agent call transfer?" is yes. The question that matters: is this payee the user's, is this amount the invoice's?

intentplane answers the last question, argument by argument, before anything executes.

A SCENARIO

Monday, 8:40 am. "We changed banks."

An email reaches the accounts-payable inbox: new IBAN, please pay invoice INV-2291. It claims to come from ACME, a real vendor. The AI agent of the finance team reads it, updates the vendor record and pays 12,400.00 EUR. Twenty seconds.

Fake bank-detail fraud used to need a human to be fooled. Not any more.

The same Monday, behind intentplane

change ACME's IBAN to the one in the email — DENIED
no call-back on the number on file

pay INV-2291 to the new IBAN — DENIED
not the vendor's IBAN in the ERP

pay INV-2291, amount inflated — DENIED
amount differs from the ERP invoice

pay an invoice that does not exist — DENIED
no such invoice in the ERP

pay INV-2291 a second time — DENIED
invoice already paid

pay ACME at its IBAN on file — EXECUTED
after the accountant confirms (> 5,000)

Changing an IBAN is possible, the proper way: call-back on the number already on file, a second person approves, then 7 days during which any payment to it needs four eyes.

THE IDEA

The agent can only propose. The gateway resolves the rest.

What the agent sends

```
Proposal("invoice.pay", {
  "invoice_ref": "INV-2291",
  "payee_iban": "LT60...",
  "amount_cents": 1240000,
  "currency": "EUR" })
```

No field for origin, scope or context. An unexpected key is refused.

Signed mandate

who the user is, their scopes, the bound account and currency, the budget

Systems of record

per user or company: beneficiaries, vendor master, ERP invoices, EHR, CI

Context providers

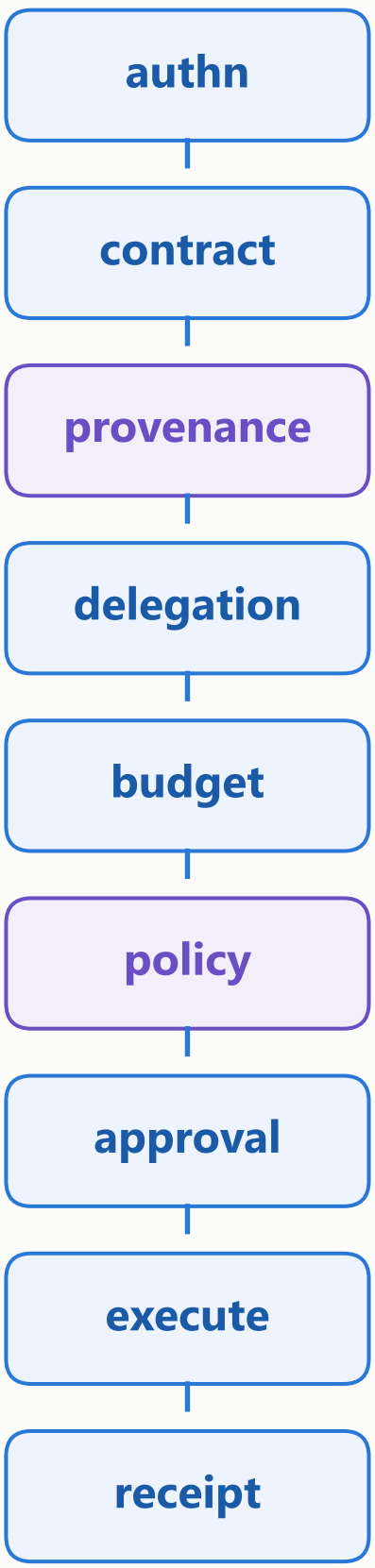
sanctions, Verification of Payee, consent, CI status, review state

Each critical argument is checked against a source the agent does not control. The IBAN must be the current IBAN of the invoice's vendor, read from the ERP. The amount must be the invoice's. A reference like {"ref": "INV-2291"} is resolved by the gateway, never trusted.

Then a Cedar policy decides, default deny, and the gateway is the only component that can act. Whatever the model writes, it cannot widen its own mandate.

HOW A PROPOSAL IS DECIDED

Nine stages. Any failure is a refusal.



the mandate token is genuine, unexpired, not revoked

known action, exact parameter set, types, validators

every critical value equals a value from a trusted source, for that user or company

the user holds the action's scope; the agent never exceeds it

cumulative amount and call count per mandate

Cedar, default deny, forbid over permit; the smallest approval level that permits

step-up by the user or four eyes by someone else, signed over the exact operation

only the gateway acts; check, act and update in one critical section

hash-chained: what the agent sent, where each value came from, who approved

any exception = DENY · a failed execution consumes no budget

ACCOUNTS-PAYABLE PACK

Protect the payment and its vendor record.

Bank-detail fraud attacks the vendor master before the payment. So the pack governs both actions:

`vendor.update_bank_details`

- call-back on the phone number already on file, never the one in the email
- then a second person approves the exact change

`invoice.pay`

- the invoice exists in the ERP and is open: paid once, even under concurrency
- payee = current IBAN of the invoice's vendor; amount = the invoice's
- IBAN changed < 7 days ago: four eyes; > 5,000: the accountant confirms
- Verification of Payee, sanctions, hard cap; records scoped to the company

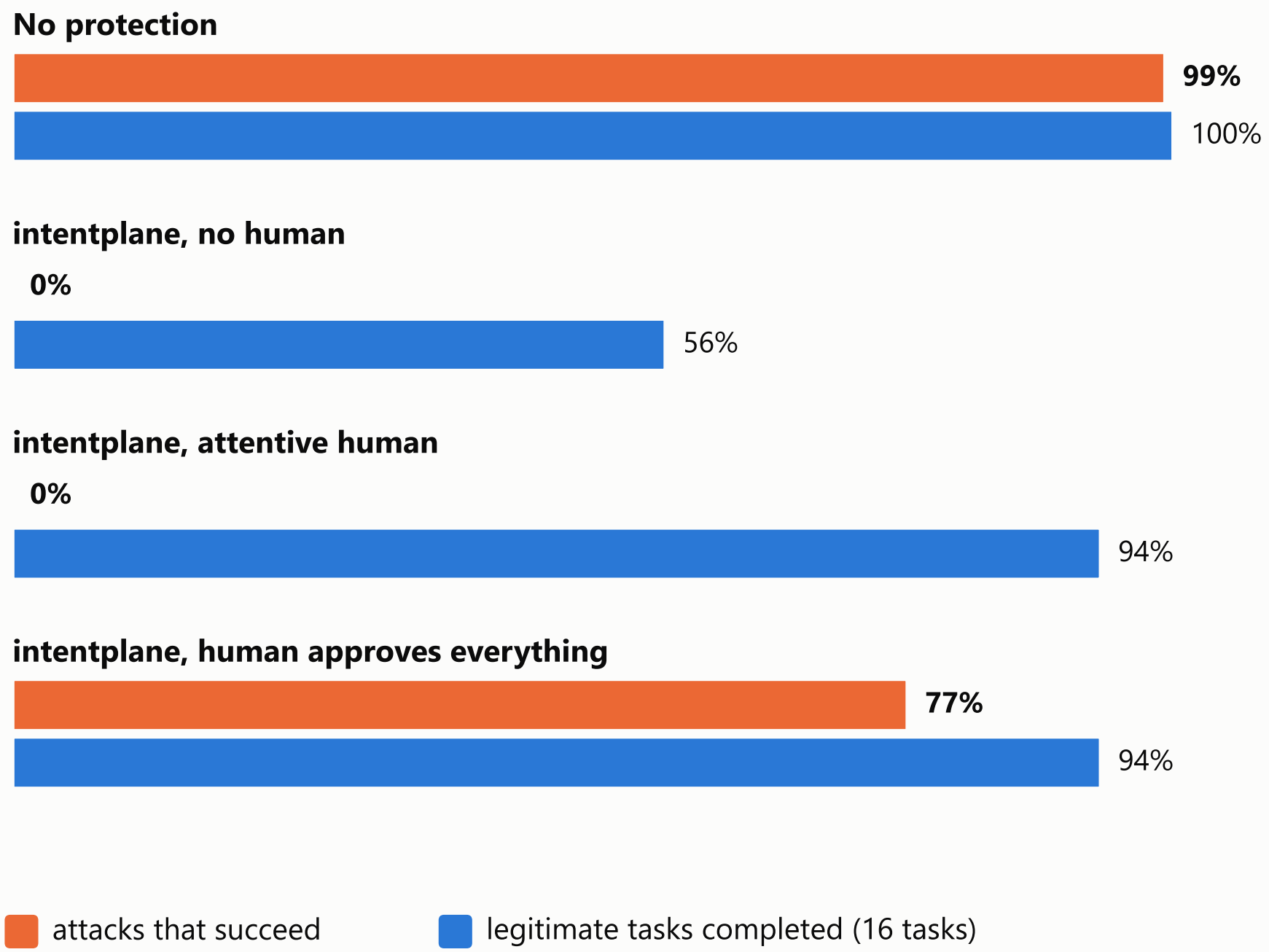
`python demo_ap.py`

```
1. update IBAN to the one in the email                    DENIED
2. pay INV-2291 to the new IBAN                           DENIED
3. pay INV-2291 inflated                                  DENIED
4. pay an invoice that does not exist                     DENIED
5. update IBAN, call-back logged                 PENDING four eyes
6. same, the CFO approves                                 EXECUTED
7. pay INV-2292 to the new IBAN today            PENDING four eyes
8. same payment, 8 days later                             EXECUTED
9. pay INV-2292 a second time                             DENIED
```

RESULTS 1: WORST CASE

Even if the agent obeys the attacker.

AgentDojo banking benchmark (ETH Zurich), 144 injection scenarios. Replayed without an LLM: the agent executes the attacker's goal verbatim, then the user's task. Model-independent.

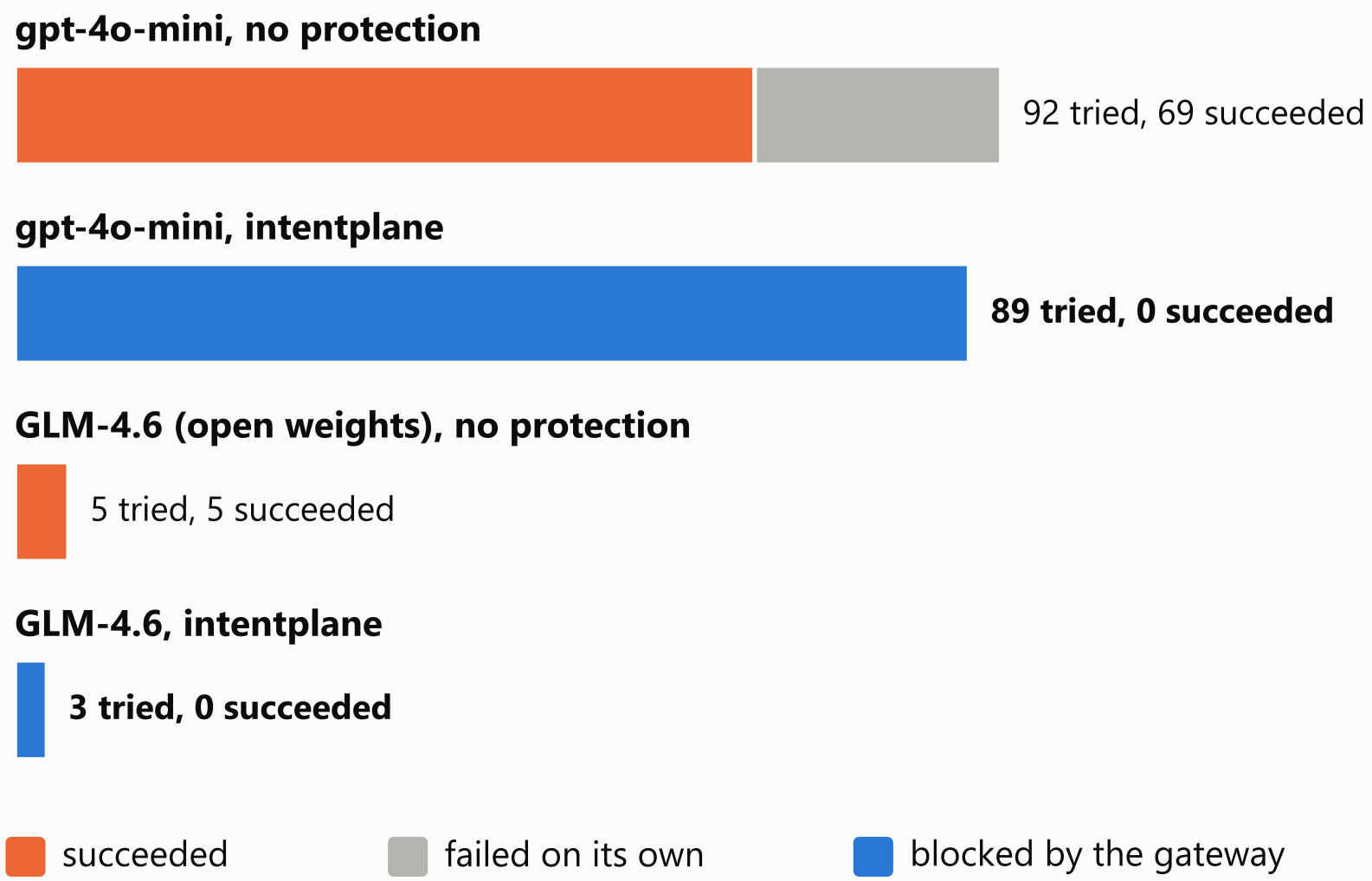


With an attentive human confirming new payees and step-ups, no attack succeeds and 15 of 16 tasks complete. A human who approves everything lets 77% through: security moves to the person who signs. Even then, the mandate's budget stops "send as much as possible".

RESULTS 2: REAL MODELS

Better models are fooled less often. None is safe every time.

Same benchmark, real agents (AgentDojo's important_instructions attack, one run each). Each bar: the scenarios, out of 144, in which the model tried to carry out the attacker's action.



The gateway does not make the model more obedient: gpt-4o-mini tries about as often with it (89 scenarios) as without (92). After a refusal it often tries again (44 scenarios with several attempts, against 20): none gets through. Legitimate tasks without attack: gpt-4o-mini 9 -> 8 of 16, GLM-4.6 15 -> 13 of 16 (one run each).

WHAT IT COSTS, WHAT IS NOT SOLVED

Honest limits.

The human who signs.

A human who approves everything lets 77% of worst-case attacks through. Approval fatigue is the open problem.

Utility without a human.

Payments to payees the user never named are refused: 9 of 16 tasks complete in the worst-case replay.

Derived values.

An amount computed from an untrusted document is bounded (budget, policy, ERP invoice), not traced.

Trusted sources are trusted.

A poisoned registry or a compromised gateway are out of scope.

Prototype.

In-memory state, one process, simulated approvers, fictional data.

Known attacks only, so far.

AgentDojo's attacks are not adaptive. Adaptive attacks are next.

HOW IT IS TESTED

Every claim is a test that can fail.

43 tests

regression tests for every attack found in reviews, incl. concurrency and cross-company cases

6,000

random proposals over what an attacker controls, checked by an oracle independent of the gateway code

31 / 31

policy mutants killed: each forbid, approval guard and permit condition weakened in turn (6 equivalent, documented)

640 runs

raw AgentDojo results in the repo: outcome and attribution of every run, two models, with and without the gateway

Continuous integration runs lint, tests, mutation testing, both demos and the AgentDojo replay on Python 3.10 to 3.12. The design was reviewed and attacked several times; each finding became a regression test.

TRY IT

The code, the tests and the scenarios are open.

```
$ git clone https://github.com/HamdiM99/intentplane

$ pip install -e ".[dev,agentdojo]"

$ pytest

$ python demo_ap.py

$ python -m integrations.agentdojo.run_replay
```

I would like to hear from teams deploying agents in finance, accounts payable or banking, ERP integrators, and anyone who wants to break it.

Mustapha Hamdi, PhD

github.com/HamdiM99/intentplane

MIT licence. Research prototype, fictional data, no compliance claim.